Beaches_Bikes_RSM

Derek Kusmenko

This notebook was produced to aid RSM students with some data analysis for their project. They mainly wanted data on Daily breakdown (weekday vs. weekend), Weekly averages, and Full-year data to assess seasonality trends.

Importing Data and Libraries

```r
library(opendatatoronto)
```

```
Warning: package 'opendatatoronto' was built under R version 4.5.2
```

```r
library(tidyverse)
```

```
Warning: package 'purrr' was built under R version 4.5.2

-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
library(sf)
```

```
Warning: package 'sf' was built under R version 4.5.2
```

```
Linking to GEOS 3.13.1, GDAL 3.11.4, PROJ 9.7.0; sf_use_s2() is TRUE
```

```r
library(leaflet)
```

```
Warning: package 'leaflet' was built under R version 4.5.2
```

```r
library(skimr)
```

```
Warning: package 'skimr' was built under R version 4.5.2
```

```r
library(janitor)
```

```
Warning: package 'janitor' was built under R version 4.5.2


Attaching package: 'janitor'

The following objects are masked from 'package:stats':

    chisq.test, fisher.test
```

```r
library(lubridate)
library(tidytext)
```

```
Warning: package 'tidytext' was built under R version 4.5.2
```

```r
set.seed(2453)
```

```r
df <- read.csv("../01_Data/beaches_bikes.csv")
```

Analysis for RSM Students

```r
df_clean <- df %>%
  mutate(
    started_at = ymd_hms(datetime_start),
    date = as.Date(started_at),
    # We recreate the year and month safely from the date object for correct sorting
    year = year(date),
    month = month(date, label = TRUE, abbr = TRUE),
    # Create Day of Week and Weekend/Weekday flag
    day_of_week = wday(date, label = TRUE, abbr = FALSE),
    day_type = ifelse(day_of_week %in% c("Saturday", "Sunday"), "Weekend", "Weekday"),
    # Create a continuous weekly identifier for multi-year trending
    year_week = floor_date(date, "week")
  ) %>%
  # Filter out any rows with missing dates that might break the plots
  filter(!is.na(date))

head(df_clean)
```

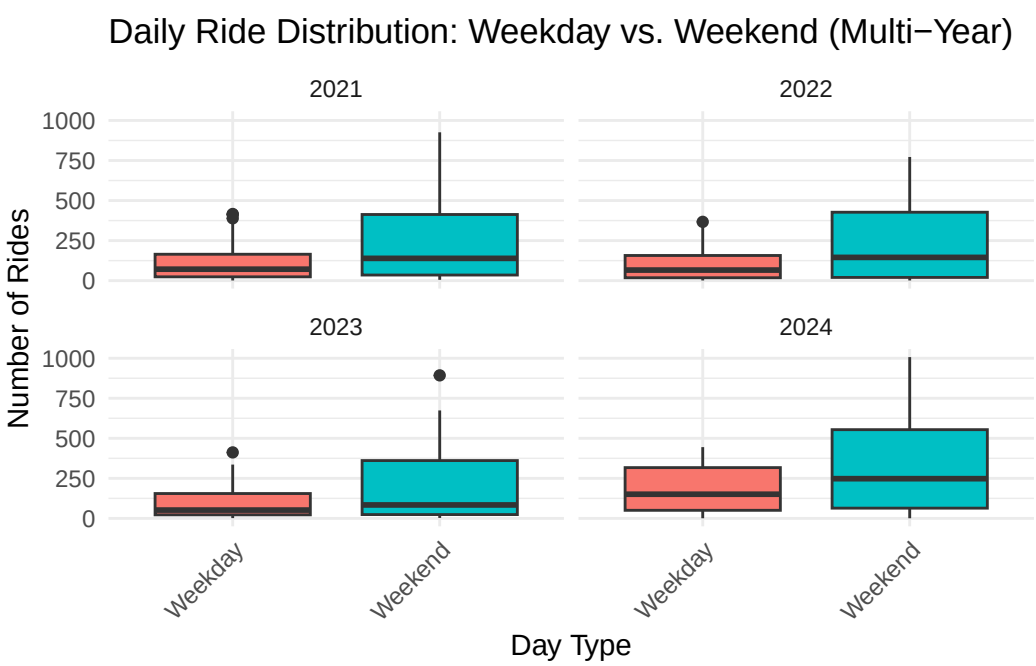
```
   Trip.Id Trip..Duration Start.Station.Id       Start.Time
1 10644802            352             7446 01/01/2021 11:12
2 10644809            570             7427 01/01/2021 11:18
3 10644836            962             7318 01/01/2021 11:28
4 10645084            997             7644 01/01/2021 13:05
5 10645129            461             7427 01/01/2021 13:23
6 10645153            961             7429 01/01/2021 13:29
                        Start.Station.Name End.Station.Id         End.Time
1                             Orchard Park           7427 01/01/2021 11:18
2 Northern Dancer Blvd / Lake Shore Blvd E           7318 01/01/2021 11:28
3           Hubbard Blvd. / Glen Manor Dr.           7354 01/01/2021 11:44
4             East Lynn Ave / Danforth Ave           7427 01/01/2021 13:21
5 Northern Dancer Blvd / Lake Shore Blvd E           7330 01/01/2021 13:30
6          Woodbine Subway Station - SMART           7427 01/01/2021 13:45
                          End.Station.Name Bike.Id     User.Type ï..Trip.Id
1 Northern Dancer Blvd / Lake Shore Blvd E    5461 Annual Member         NA
2           Hubbard Blvd. / Glen Manor Dr.    5461 Annual Member         NA
3 Tommy Thompson Park (Leslie Street Spit)    5461 Annual Member         NA
4 Northern Dancer Blvd / Lake Shore Blvd E    3347 Casual Member         NA
5                     Lee Ave / Queen St E    3347 Casual Member         NA
6 Northern Dancer Blvd / Lake Shore Blvd E    2337 Annual Member         NA
  Model       datetime_start start_date_actual start_hour_of_day
1  <NA> 2021-01-01T11:12:00Z        2021-01-01                11
2  <NA> 2021-01-01T11:18:00Z        2021-01-01                11
```

```
3  <NA> 2021-01-01T11:28:00Z        2021-01-01                11
4  <NA> 2021-01-01T13:05:00Z        2021-01-01                13
5  <NA> 2021-01-01T13:23:00Z        2021-01-01                13
6  <NA> 2021-01-01T13:29:00Z        2021-01-01                13
  start_month_name year         datetime_end end_date_actual end_hour_of_day
1              Jan 2021 2021-01-01T11:18:00Z      2021-01-01              11
2              Jan 2021 2021-01-01T11:28:00Z      2021-01-01              11
3              Jan 2021 2021-01-01T11:44:00Z      2021-01-01              11
4              Jan 2021 2021-01-01T13:21:00Z      2021-01-01              13
5              Jan 2021 2021-01-01T13:30:00Z      2021-01-01              13
6              Jan 2021 2021-01-01T13:45:00Z      2021-01-01              13
  end_month_name          started_at       date month day_of_week day_type
1            Jan 2021-01-01 11:12:00 2021-01-01   Jan      Friday  Weekday
2            Jan 2021-01-01 11:18:00 2021-01-01   Jan      Friday  Weekday
3            Jan 2021-01-01 11:28:00 2021-01-01   Jan      Friday  Weekday
4            Jan 2021-01-01 13:05:00 2021-01-01   Jan      Friday  Weekday
5            Jan 2021-01-01 13:23:00 2021-01-01   Jan      Friday  Weekday
6            Jan 2021-01-01 13:29:00 2021-01-01   Jan      Friday  Weekday
   year_week
1 2020-12-27
2 2020-12-27
3 2020-12-27
4 2020-12-27
5 2020-12-27
6 2020-12-27
```

```r
#  Define outlier function
is_outlier <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  return(x < (q1 - 1.5 * iqr) | x > (q3 + 1.5 * iqr))
}
```

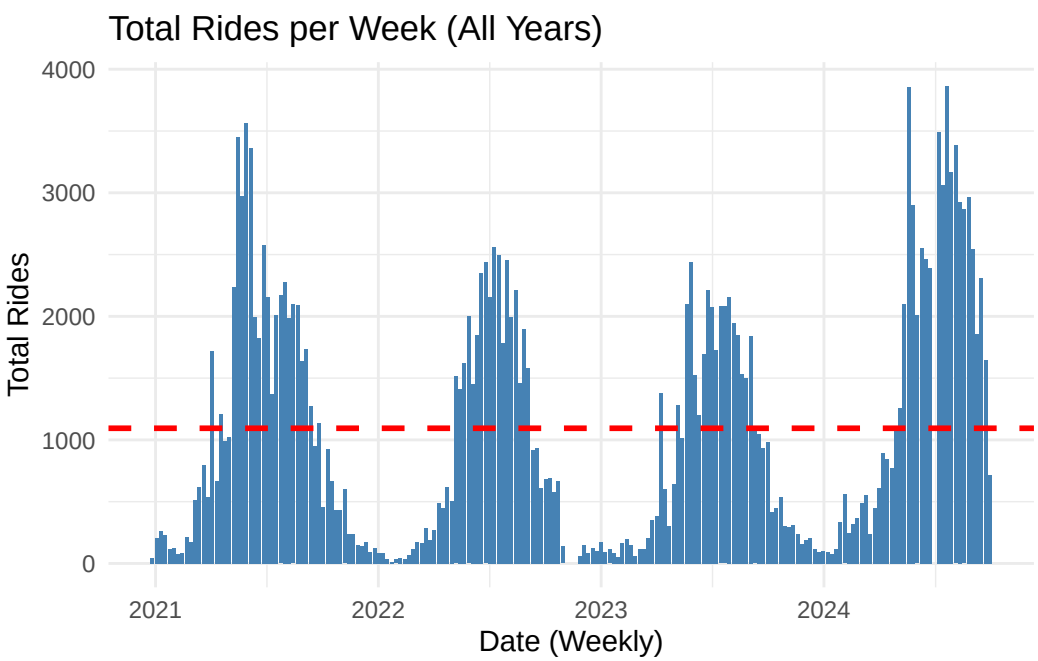
```r
# Daily Breakdown
daily_counts_clean <- df_clean %>%
  group_by(date, day_type, year) %>%
  summarise(total_rides = n(), .groups = 'drop') %>%
  group_by(day_type) %>%
  filter(!is_outlier(total_rides)) %>%
  ungroup()
```

```r
# plot the boxplots faceted by year
ggplot(daily_counts_clean, aes(x = day_type, y = total_rides, fill = day_type)) +
  geom_boxplot() +
  facet_wrap(~ year) +
  labs(title = "Daily Ride Distribution: Weekday vs. Weekend (Multi-Year)",
       y = "Number of Rides",
       x = "Day Type") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "none")
```

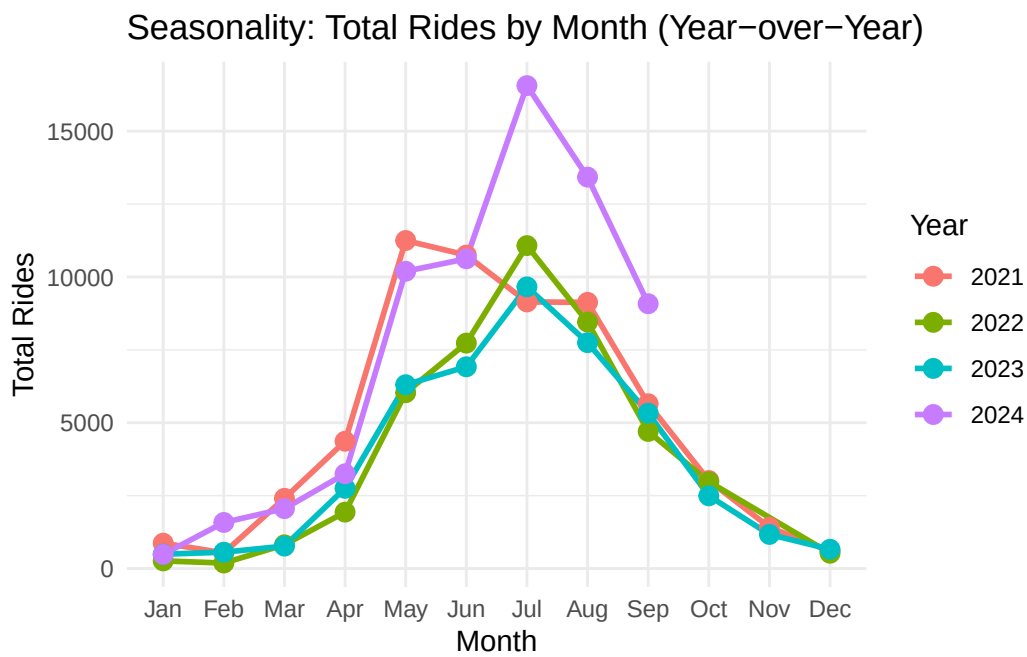



```r
# Weekly Averages
weekly_data_clean <- df_clean %>%
  group_by(year_week) %>%
  summarise(total_weekly_rides = n(), .groups = 'drop') %>%
  filter(!is_outlier(total_weekly_rides))

overall_weekly_avg <- mean(weekly_data_clean$total_weekly_rides)

# plot chart over all years
ggplot(weekly_data_clean, aes(x = year_week, y = total_weekly_rides)) +
  geom_col(fill = "steelblue") +
```

```
  geom_hline(yintercept = overall_weekly_avg, linetype = "dashed", color = "red", linewidth =
  labs(title = "Total Rides per Week (All Years)",
       x = "Date (Weekly)",
       y = "Total Rides") +
  theme_minimal()
```

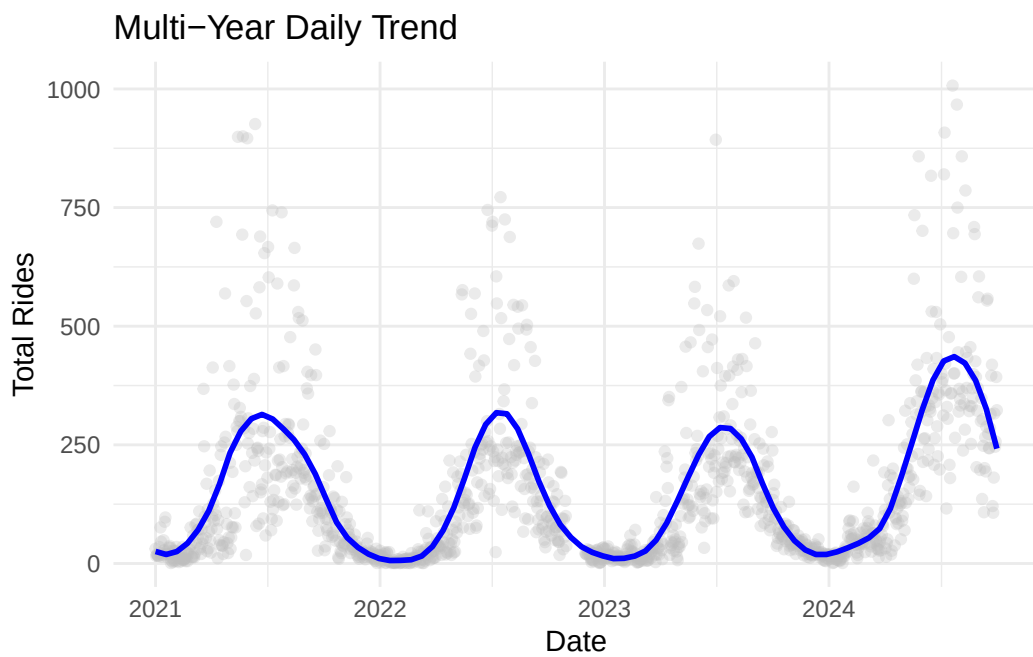



```
# seasonality trends
monthly_trend_clean <- df_clean %>%
  group_by(year, month) %>%
  summarise(total_rides = n(), .groups = 'drop') %>%
  filter(!is_outlier(total_rides))

# plot
ggplot(monthly_trend_clean, aes(x = month, y = total_rides, group = year, color = as.factor(y
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Seasonality: Total Rides by Month (Year-over-Year)",
       x = "Month",
       y = "Total Rides",
       color = "Year") +
  theme_minimal()
```




```r
# multi year daily trend with smoothing
ggplot(daily_counts_clean, aes(x = date, y = total_rides)) +
  geom_point(alpha = 0.3, color = "gray") +
  geom_smooth(method = "loess", color = "blue", se = FALSE, span = 0.15) +
  labs(title = "Multi-Year Daily Trend",
       x = "Date",
       y = "Total Rides") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
# Exporting the general analysis data
write.csv(daily_counts_clean, "daily_ride_distribution.csv", row.names = FALSE)
write.csv(weekly_data_clean, "weekly_ride_averages.csv", row.names = FALSE)
write.csv(monthly_trend_clean, "monthly_seasonality_trends.csv", row.names = FALSE)
```

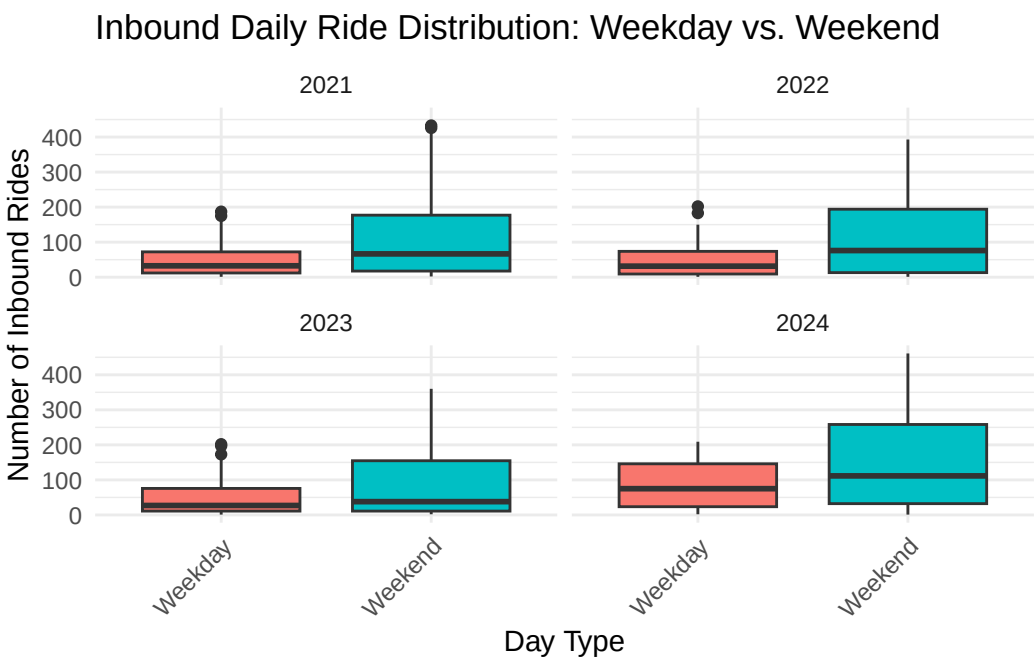
Repeat Analysis for Inbound Trips Only

```
# variable for beaches stations only
beaches_stations <- c(
  "7692",
  "7317",
  "7318",
  "7365",
  "7427",
  "7315",
  "7695",
  "7364",
  "7314",
  "7316",
  "7428"
)
```

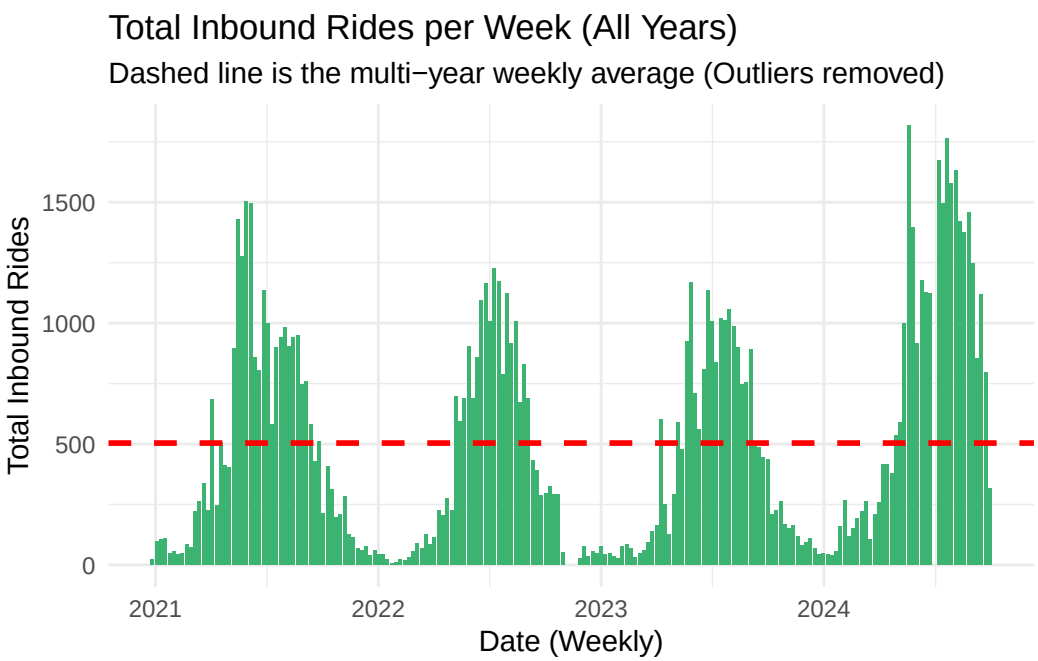
```r
# Create the inbound filtered dataset
df_inbound <- df %>%
  # trip ends in beaches
  filter(End.Station.Id %in% beaches_stations) %>%
  # trip starts outside of beaches
  filter(!(Start.Station.Id %in% beaches_stations))
```

```r
# preprocess data
df_clean <- df_inbound %>%
  mutate(

    started_at = ymd_hms(datetime_start),
    date = as.Date(started_at),

    year = year(date),
    month = month(date, label = TRUE, abbr = TRUE),

    day_of_week = wday(date, label = TRUE, abbr = FALSE),
    day_type = ifelse(day_of_week %in% c("Saturday", "Sunday"), "Weekend", "Weekday"),

    year_week = floor_date(date, "week")
  ) %>%
  filter(!is.na(date))
```

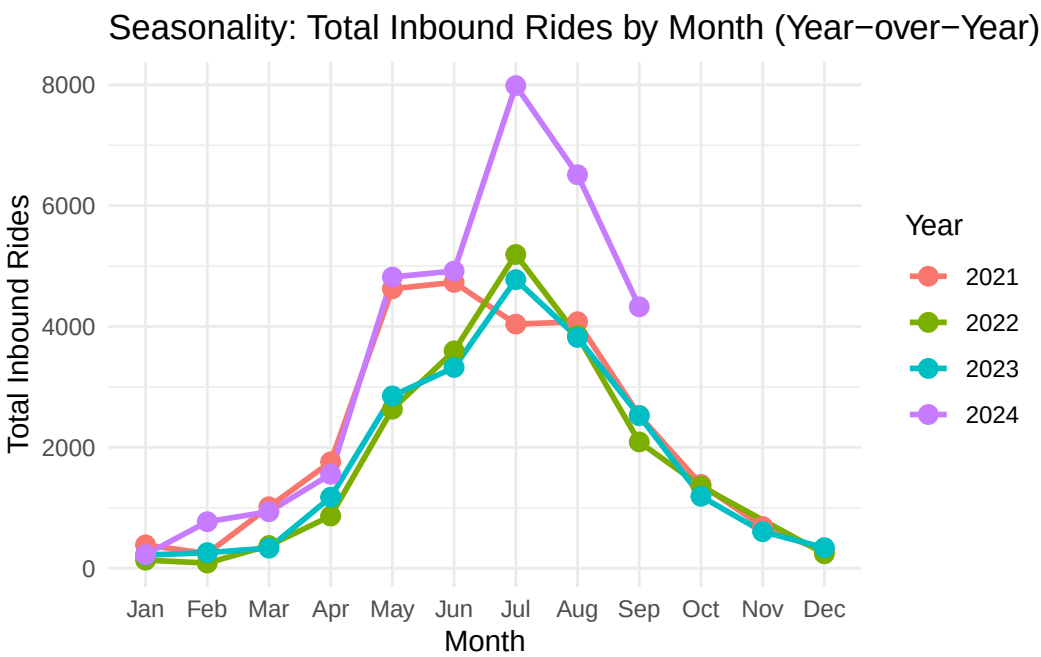
```r
# daily breakdown
daily_counts_clean <- df_clean %>%
  group_by(date, day_type, year) %>%
  summarise(total_rides = n(), .groups = 'drop') %>%
  group_by(day_type) %>%
  filter(!is_outlier(total_rides)) %>%
  ungroup()

ggplot(daily_counts_clean, aes(x = day_type, y = total_rides, fill = day_type)) +
  geom_boxplot() +
  facet_wrap(~ year) +
  labs(title = "Inbound Daily Ride Distribution: Weekday vs. Weekend",
       y = "Number of Inbound Rides",
       x = "Day Type") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "none")
```




```r
# weekly averages
weekly_data_clean <- df_clean %>%
  group_by(year_week) %>%
  summarise(total_weekly_rides = n(), .groups = 'drop') %>%
  filter(!is_outlier(total_weekly_rides))

overall_weekly_avg <- mean(weekly_data_clean$total_weekly_rides)

ggplot(weekly_data_clean, aes(x = year_week, y = total_weekly_rides)) +
  geom_col(fill = "mediumseagreen") +
  geom_hline(yintercept = overall_weekly_avg, linetype = "dashed", color = "red", linewidth =
  labs(title = "Total Inbound Rides per Week (All Years)",
       subtitle = "Dashed line is the multi-year weekly average (Outliers removed)",
       x = "Date (Weekly)",
       y = "Total Inbound Rides") +
  theme_minimal()
```

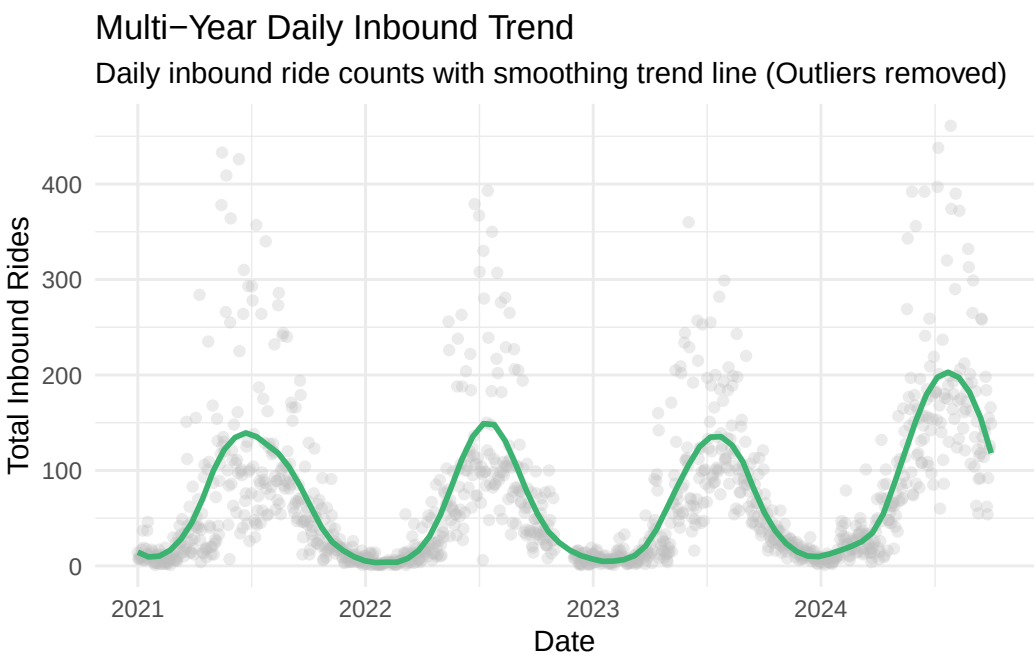



```
# seasonality trends
monthly_trend_clean <- df_clean %>%
  group_by(year, month) %>%
  summarise(total_rides = n(), .groups = 'drop') %>%
  filter(!is_outlier(total_rides))

ggplot(monthly_trend_clean, aes(x = month, y = total_rides, group = year, color = as.factor(y
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Seasonality: Total Inbound Rides by Month (Year-over-Year)",
       x = "Month",
       y = "Total Inbound Rides",
       color = "Year") +
  theme_minimal()
```




```
# multi year daily trend with smoothing
ggplot(daily_counts_clean, aes(x = date, y = total_rides)) +
  geom_point(alpha = 0.3, color = "gray") +
  geom_smooth(method = "loess", color = "mediumseagreen", se = FALSE, span = 0.15) +
  labs(title = "Multi-Year Daily Inbound Trend",
       subtitle = "Daily inbound ride counts with smoothing trend line (Outliers removed)",
       x = "Date",
       y = "Total Inbound Rides") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
# Exporting the general analysis data
write.csv(daily_counts_clean, "daily_ride_distribution_inbound.csv", row.names = FALSE)
write.csv(weekly_data_clean, "weekly_ride_averages_inbound.csv", row.names = FALSE)
write.csv(monthly_trend_clean, "monthly_seasonality_trends_inbound.csv", row.names = FALSE)
```